```
import numpy as np

class WarehouseMDP:
    def __init__(self):
        self.states = ['LOW_A', 'LOW_B', 'HIGH_A', 'HIGH_B']
        self.actions = ['PICK', 'MOVE', 'CHARGE']

        # Transition Model: P(next_state | state, action)
        self.transitions = {
            'LOW_A': {
                'PICK': [('HIGH_A', 1.0)],
                'MOVE': [('LOW_B', 1.0)],
                'CHARGE': [('LOW_A', 1.0)]
            },
            'LOW_B': {
                'PICK': [('HIGH_B', 1.0)],
                'MOVE': [('LOW_A', 1.0)],
                'CHARGE': [('LOW_B', 1.0)]
            },
            'HIGH_A': {
                'PICK': [('HIGH_A', 1.0)],   # already loaded
                'MOVE': [('HIGH_B', 1.0)],
                'CHARGE': [('LOW_A', 1.0)]
            },
            'HIGH_B': {
                'PICK': [('HIGH_B', 1.0)],
                'MOVE': [('HIGH_A', 1.0)],
                'CHARGE': [('LOW_B', 1.0)]
            }
        }

        # Rewards (efficiency-based)
        self.rewards = {
            'LOW_A': 5,
            'LOW_B': 5,
            'HIGH_A': 10,
            'HIGH_B': 10
        }


# ---------------- POLICY ITERATION ----------------
def policy_iteration(mdp, gamma=0.9, theta=0.01):

    # Step 1: Initialize random policy
    policy = {s: np.random.choice(mdp.actions) for s in mdp.states}
    V = {s: 0 for s in mdp.states}

    while True:

        # -------- Policy Evaluation --------
        while True:
            delta = 0
            for state in mdp.states:
                v = V[state]
                action = policy[state]

                V[state] = sum(
                    prob * (mdp.rewards[next_state] + gamma * V[next_state])
                    for next_state, prob in mdp.transitions[state][action]
                )

                delta = max(delta, abs(v - V[state]))
```

```
            if delta < theta:
                break

        # -------- Policy Improvement --------
        policy_stable = True

        for state in mdp.states:
            old_action = policy[state]

            # Find best action
            action_values = {}
            for action in mdp.actions:
                action_values[action] = sum(
                    prob * (mdp.rewards[next_state] + gamma * V[next_state])
                    for next_state, prob in mdp.transitions[state][action]
                )

            best_action = max(action_values, key=action_values.get)
            policy[state] = best_action

            if old_action != best_action:
                policy_stable = False

        if policy_stable:
            break

    return policy, V


# ---------------- MAIN ----------------
if __name__ == "__main__":
    mdp = WarehouseMDP()

    policy, values = policy_iteration(mdp)

    print("Optimal State Values:")
    for state, value in values.items():
        print(f"{state}: {round(value, 2)}")

    print("\nOptimal Policy:")
    for state, action in policy.items():
        print(f"{state} → {action}")
```

```
Optimal State Values:
LOW_A: 99.93
LOW_B: 99.93
HIGH_A: 99.93
HIGH_B: 99.93

Optimal Policy:
LOW_A → PICK
LOW_B → PICK
HIGH_A → PICK
HIGH_B → PICK
```